Tarea 2 — Procedimientos y clausuras

Unidad 2: Procedimientos y clausuras

Lenguajes de Programación · Semestre 2026-2
Lic. en Ciencias de la Computación · Universidad de Sonora

> Implementación en `unidad-2/tarea2/`. Al correr `make test` pasan 378 pruebas en `proc-varios-nucleo`, 373 en `proc-varios-azucar` y 363 en `proc-dyn`.

1. Procedimientos de varios parámetros

1.1 Núcleo: reglas de inferencia

Llamo `(call-exp e0 (e1 ... en))` a una llamada cuyo operador es `e0` y cuyos `n` operandos son `e1, ..., en`. La lista de parámetros de un procedimiento es `xs`. Para extender el entorno con varios nombres a la vez uso la notación `[xs=ℓ]ρ` de la hoja de referencia.

[proc]

$$\frac{}{\text{(value-of (proc-exp xs c) } \rho\text{)} = \text{clausura(xs, c, } \rho\text{)}}$$

[call]

$$\frac{\begin{array}{c}\text{(value-of e0 } \rho\text{)} = \text{clausura(xs, c, } \rho'\text{)} \\ \text{(value-of ei } \rho\text{)} = \text{vi} \quad \text{para i = 1, ..., n} \\ |\text{xs}| = \text{n}\end{array}}{\text{(value-of (call-exp e0 (e1 ... en)) } \rho\text{)} = \text{(value-of c [xs=(v1 ... vn)]}\rho'\text{)}}$$

[call-err]

$$\frac{\begin{array}{c}\text{(value-of e0 } \rho\text{)} = \text{clausura(xs, c, } \rho'\text{)} \\ \text{(value-of ei } \rho\text{)} = \text{vi} \quad \text{para i = 1, ..., n} \\ |\text{xs}| \neq \text{n}\end{array}}{\text{(value-of (call-exp e0 (e1 ... en)) } \rho\text{)} = \text{error}}$$

mensaje: `apply-procedure: número de argumentos equivocado: esperaba |xs| y me diste n`

En `[call]` y `[call-err]` las premisas se leen de arriba hacia abajo y de izquierda a derecha: primero se evalúa el operador y después los operandos, todos en el entorno de la llamada `ρ`. La conclusión de `[call]` usa `ρ'`, el entorno que quedó guardado en la clausura, extendido con todos los parámetros al mismo tiempo mediante `[xs=(v1 ... vn)]ρ'`. Si algún nombre se repite, se queda con la última vinculación, igual que pasa con `unpack`. Cuando `e0` no evalúa a un procedimiento, el error que sale es el de `val->proc`; `[call-err]` solo cubre el caso en el que sí es un procedimiento pero las longitudes no coinciden.

1.2 Varios parámetros como azúcar: reglas de transformación

Escribo `proc*(x, y, xs ...) c` como abreviatura de `(proc*-exp (x y . xs) c)`, y `call*(e0, e1, e2, es ...)` como abreviatura de `(call*-exp e0 (e1 e2 . es))`. Las formas finales, con un solo parámetro o un solo operando, solo aparecen como paso intermedio de la traducción; en realidad ya son el núcleo.

[proc*]

$$\frac{}{\text{proc*(x, y, xs ...) c} \rightsquigarrow \text{proc (x) proc*(y, xs ...) c}}$$

[proc*-fin]

$$\frac{}{\text{proc*(x) c} \rightsquigarrow \text{proc (x) c}}$$

[call*]

$$\frac{}{\text{call*(e0, e1, e2, es ...)} \rightsquigarrow \text{call*((e0 e1), e2, es ...)}}$$

[call*-fin]

$$\frac{}{\text{call*(e0, e1)} \rightsquigarrow \text{(e0 e1)}}$$

Cada vez que aplico `[proc*]` quito el primer parámetro y dejo una forma con uno menos; con `[call*]` pasa lo mismo con el primer operando, y queda una llamada anidada. El resultado se vuelve a pasar por `desugar`. La anidación de `[call*]` va hacia la izquierda: `(f a b c)` se convierte en `(((f a) b) c)`, de modo que el operador y los operandos se evalúan de izquierda a derecha, igual que en el núcleo.

1.3 Comparación: núcleo frente a azúcar

Estos son los resultados de correr los cinco programas en las dos carpetas:

#	Programa	`proc-varios-nucleo`	`proc-varios-azucar`
P1	`let resta = proc (x, y) -(x, y) in (resta 10 3)`	7	7
P2	`let resta = proc (x, y) -(x, y) in let quita10 = (resta 10) in (quita10 3)`	apply-procedure: número de argumentos equivocado: esperaba 2 y me diste 1	7
P3	`let resta = proc (x, y) -(x, y) in (resta 10 3 1)`	apply-procedure: número de argumentos equivocado: esperaba 2 y me diste 3	val->proc: Esperaba un procedimiento pero me diste 7
P4	`(proc () 5)`	5	parse: no esperaba) después de (
P5	`let f = proc (x) proc (y) -(x, y) in (f 10 3)`	apply-procedure: número de argumentos equivocado: esperaba 1 y me diste 2	7

Lo que gana y lo que pierde cada enfoque

El núcleo permite declarar procedimientos con el número exacto de parámetros que se escribe, incluido el caso de cero parámetros. P4 vale 5 porque `proc () 5` es una clausura con lista vacía y `(proc () 5)` la llama con una lista vacía. Además, cuando el número de operandos no coincide, el error aparece en la misma llamada y dice cuántos argumentos esperaba y cuántos recibió: en P3 sale `esperaba 2 y me diste 3`.

Lo que se pierde con el núcleo es la aplicación parcial y la posibilidad de combinar con procedimientos currificados. En P2, `(resta 10)` no es un procedimiento que espera el segundo argumento: es una llamada con un solo operando a un procedimiento de dos parámetros, así que termina en error de aridad. En P5, un

procedimiento currificado, que es justo el que sí permite aplicación parcial, no se puede llamar con dos operandos: `(f 10 3)` da `esperaba 1 y me diste 2`.

El azúcar permite la aplicación parcial y la currificación sin tocar el evaluador. P2 vale 7 porque `(resta 10)` termina siendo el `proc (y) -(10, y)` que queda después de aplicar el primer `proc`, y `(quita10 3)` lo aplica. P5 también vale 7: el procedimiento currificado se llama con dos operandos porque la llamada se anida en dos llamadas de un solo operando.

Con el azúcar se pierden los procedimientos sin parámetros y el diagnóstico inmediato cuando sobran operandos. P4 ni siquiera se parsea, porque la producción de `proc*` pide al menos un parámetro (y la de `call*`, al menos dos operandos). En P3 la llamada con tres operandos es válida: se traduce a `(((resta 10) 3) 1)`, las dos primeras llamadas dan 7 y el error sale después, al intentar aplicar 7 como procedimiento: `val->proc: Esperaba un procedimiento pero me diste 7`. También se pierde la forma original en el REPL, como se ve en la pregunta 3.

Las tres preguntas

1. ¿Qué es `(resta 10)` en cada enfoque?

En el núcleo es una llamada con un solo operando a un procedimiento de dos parámetros: no hay regla que la reduzca y da error de aridad (P2). Con el azúcar es un procedimiento de un parámetro, el que espera el segundo argumento; por eso `(quita10 3)` vale 7 (P2).

2. ¿Cuándo se detecta un operando de más y con qué mensaje?

En el núcleo se detecta en la llamada misma, antes de evaluar el cuerpo, con `apply-procedure: número de argumentos equivocado: esperaba 2 y me diste 3` (P3). Con el azúcar no se detecta como error de aridad: la llamada se acepta y se traduce a llamadas anidadas. Después de obtener 7, la llamada extra intenta usar ese 7 como operador y falla con `val->proc: Esperaba un procedimiento pero me diste 7` (P3).

3. ¿Qué escribe el REPL para `proc (x, y) x` en cada uno, y por qué?

En el núcleo escribe `<proc (x, y)>`, porque la clausura guarda `xs` y `expval->string` recorre todos los parámetros. Con el azúcar escribe `<proc (x)>`, porque el texto se tradujo a `proc (x) proc (y) x` y lo que se imprime es el `proc` exterior, que solo tiene el parámetro `x` y todavía no se ha aplicado. El azúcar no conserva la forma de la superficie.

Cambios por carpeta (parte 1)

proc-varios-nucleo: en `ast.rkt`, `proc-exp` pasó de `x` a `xs` y `call-exp` de `e2` a `es`. En `parser.rkt` agregué los no terminales `parameters`, `nonempty-parameters`, `operands` y `nonempty-operands`, junto con las producciones `proc (Parameters) Expression` y `(Expression Operands)`. En `desugar.rkt` las dos cláusulas del núcleo reconstruyen `xs` y traducen cada operando con `map`. En `procs.rkt` la clausura guarda `xs`, y en `vals.rkt` se imprimen todos los parámetros. En `interp.rkt`, `[proc]` construye `clausura(xs, c, ρ)`; `[call]` evalúa el operador y luego los operandos de izquierda a derecha con `value-of-operands`; `apply-procedure` recibe la lista de valores, revisa la longitud, emite el mensaje de `[call-err]` y extiende el entorno de creación con `extend-env*`.

proc-varios-azucar: en `ast.rkt` agregué `proc*-exp (xs, e1)` y `call*-exp (e1, es)`, y los incluí en `expression?`. En `parser.rkt` solo agregué las dos producciones de azúcar y los no terminales `nonempty-parameters` y `nonempty-operands`; las producciones del núcleo quedaron intactas. En `desugar.rkt` agregué las dos cláusulas que traducen a `proc` anidados y a llamadas anidadas. `interp.rkt`, `procs.rkt` y

`vals.rkt` no se tocaron: toda la diferencia está en `desugar` y el evaluador sigue siendo el de un solo parámetro.

2. Vinculación dinámica

2.1 Especificación

La sintaxis es la de PROC: un parámetro y un operando. Ahora un procedimiento es `proc(x, c)`: solo el parámetro y el cuerpo, sin entorno guardado.

[proc]

$$\frac{}{\text{(value-of (proc-exp x c) } \rho\text{)} = \text{proc(x, c)}}$$

[call]

$$\frac{\text{(value-of e1 } \rho\text{)} = \text{proc(x, c)} \qquad \text{(value-of e2 } \rho\text{)} = \text{v2}}{\text{(value-of (call-exp e1 e2) } \rho\text{)} = \text{(value-of c [x=v2]}\rho\text{)}}$$

La única diferencia con PROC está en la conclusión de `[call]`: PROC evalúa el cuerpo en `[x=v2]ρ'`, donde `ρ'` es el entorno que guarda la clausura; aquí se evalúa en `[x=v2]ρ`, el entorno desde donde se hizo la llamada. Por eso `[proc]` ya no guarda `ρ`: el valor de un procedimiento solo necesita el parámetro y el cuerpo.

2.2 Implementación

En `proc-dyn`, `interp.rkt` cambia `[proc]` para construir la clausura sin `ρ` y `[call]` para pasarle `ρ` a `apply-procedure`; `apply-procedure` evalúa el cuerpo en `(extend-env x v ρ)`, el entorno de la llamada. `procs.rkt` define `closure` con solo `x` y `body`, y `vals.rkt` ajusta `expval->string` a esos dos campos. `ast.rkt`, `parser.rkt` y `desugar.rkt` no cambian: la sintaxis es la misma y lo que cambia es la semántica.

2.3 Análisis: PROC frente a vinculación dinámica

Resultados de ejecutar D1–D5 en `proc-02-procedimientos` y en `proc-dyn`:

#	Programa	`proc-02-procedimientos` (léxico)	`proc-dyn` (dinámico)

D1	`let a = 3`
	`in let p = proc (z) a`
	`  in let f = proc (x) (p 0)`
	`  in let a = 5`
	`  in (f 2)`
		3	5

D2	`let a = 3`
	`in let p = proc (z) a`
	`  in let f = proc (a) (p 0)`
	`  in let a = 5`
	`  in (f 2)`
		3	2

D3	`let x = 200`
	`in let f = proc (z) -(z, x)`
	`  in let x = 100`
	`  in let g = proc (z) -(z, x)`
	`  in -((f 1), (g 1))`
		-100	0

#	Programa	`proc-02-procedimientos` (léxico)	`proc-dyn` (dinámico)
D4	`let menos = proc (n) proc (x) -(x, n)` `in ((menos 5) 12)`	7	apply-env: la variable n no está vinculada

D5	`let f = proc (x) -(x, y)`
	`in let y = 10`
	`  in (f 15)`
		apply-env: la variable y no está vinculada	5

Lo que gana y lo que pierde cada diseño

El alcance léxico permite que un procedimiento devuelva otro procedimiento con estado privado: en D4, el `proc (x)` interior conserva `n=5` y el resultado es 7. También hace que dos procedimientos con el mismo texto pero creados en entornos distintos sean independientes: en D3, `f` conserva `x=200` y `g` conserva `x=100`, y el resultado es −100. Y el nombre del parámetro de `f` no cambia el significado de `p`: D1 y D2 valen 3 en los dos casos.

Lo que no se puede hacer con alcance léxico es que un procedimiento use vinculaciones que solo existen en el lugar desde donde se le llama. En D5, `y` se vincula después de crear `f` y `f` no la ve: termina en `apply-env: la variable y no está vinculada`. Para conseguir lo que D5 hace con vinculación dinámica habría que pasarle `y` explícitamente a `f` o ponerla en un entorno global.

La vinculación dinámica permite que un procedimiento lea el entorno de quien lo llama sin recibir esos valores como argumentos: en D5, `f` encuentra `y=10` en la llamada y vale 5; en D1 encuentra `a=5`. El mismo procedimiento puede comportarse distinto según el entorno del llamador, y su valor no necesita guardar un entorno.

Con vinculación dinámica se pierden la encapsulación de los procedimientos que devuelven procedimientos y la independencia respecto del llamador. En D4, al llamar `(menos 5)` se obtiene un procedimiento, pero cuando se le llama después `n` ya no existe: `apply-env: la variable n no está vinculada`. En D3, `f` y `g` se llaman en el mismo entorno, donde `x=100`, así que los dos ven 100 y el resultado es 0 en vez de −100: se pierde la diferencia entre dos clausuras creadas en momentos distintos. Y en D1/D2 basta cambiar el nombre del parámetro de `f` para que el resultado pase de 3 a 5 y a 2.

Las tres preguntas

1. ¿Por qué en D1 y D2 cambiar el nombre de un parámetro cambia el resultado con vinculación dinámica y no en PROC?

En D1, el cuerpo de `p` es la variable libre `a`. Con alcance léxico, `p` guarda el entorno de su creación, donde `a=3`, y el nombre del parámetro de `f` no interviene: D1 y D2 valen 3. Con vinculación dinámica, `a` se busca en el entorno de la llamada a `p`, que es el cuerpo de `f`. En D1 el parámetro de `f` se llama `x`, así que no tapa la `a` del `let a = 5` y `p` ve 5. En D2 el parámetro se llama `a`; al llamar `(f 2)`, ese parámetro vale 2 y tapa la `a` exterior, así que `p` ve 2. Quien escribe `f` tendría que conocer todos los nombres libres de `p`, y de cualquier procedimiento que `f` llame, para saber si sus propios parámetros o variables locales los van a capturar; la interfaz de `f` deja de ser solo su texto y pasa a incluir los nombres del entorno del llamador.

2. ¿Qué le pasa a un procedimiento que devuelve otro, como en D4?

En PROC, el procedimiento interior conserva el entorno del `proc (n)` que lo creó, incluido `n=5`; al aplicarlo a 12 se evalúa `-(12, 5)` y da 7. En `proc-dyn`, el procedimiento interior no guarda ningún entorno; cuando se le

llama desde fuera, `n` no está vinculada en el entorno de la llamada y el programa falla con `apply-env: la variable n no está vinculada`. La vinculación dinámica no puede representar por sí sola el estado que captura una clausura.

3. ¿Por qué el alcance léxico es la norma?

Porque el significado de un procedimiento depende solo de su texto y de su entorno de creación: se puede razonar localmente, renombrar un parámetro sin cambiar el comportamiento (D1 y D2 dan 3), compilar por separado y devolver clausuras que conservan estado (D4 da 7). Con vinculación dinámica, en cambio, renombrar el parámetro de `f` de `x` a `a` cambia el resultado de `p` de 5 a 2, y un procedimiento devuelto pierde `n` (D4). Que cualquier cambio de nombre local en un llamador pueda alterar sin aviso al procedimiento llamado es una consecuencia concreta de por qué el alcance dinámico no sirve como regla general: hace frágil la composición y difícil el análisis estático. Se usa en casos acotados, como parámetros de configuración dinámicos, pero no como alcance por defecto.